

QuantumShield — Existing Tool Comparison

Purpose: Evidence base for why we build our own lightweight discovery scanner instead of solely relying on existing CBOM tooling.

Date: 2026-09-20

Status: Research complete; “Real observed gaps” row to be filled after Person B runs the tools.

Summary Table

Tool	Languages Supported	What it Detects	Known Limitations	Licensing / Overhead	Fit for QuantumShield
<code>cdxgen</code> (<code>--include-crypto / cbom</code> alias)	Java (primary), Python, limited JS/TS source algorithms; also .NET via dosai in newer versions	Crypto libraries + some algorithm inventory; evidence mode available; produces CycloneDX 1.6/1.7 CBOM components	Incomplete Python coverage (known issues missing many pyca / PyCryptodome / etc. call sites); originally Java-centric; constant-propagation depth varies; not pure line-level regex style	Apache-2.0; Node.js runtime; relatively heavy for a pure discovery PoC	Good component inventory but incomplete as the discovery engine targets ECDSA/AES/samples PoC
<code>sonar-cryptography</code> (CBOMkit-hyperion)	Java (JCA + BouncyCastle LW API – claimed 100 %), Python (pyca/cryptography)	Precise AST/rule-based detection of library API usage; emits full CBOM with evidence.occurrences (file + line + additionalContext)	Requires SonarQube server + quality profile activation; heavier operational	Apache-2.0 (PQCA / IBM origin); needs SonarQube	Excellent precision the engine is already SonarQube overkill

Tool	Languages Supported	What it Detects	Known Limitations	Licensing / Overhead	Fit for Purpose / Quantification
	– claimed 100 %), Go (std crypto + partial x/crypto), C# in development		footprint; language coverage still expanding; not a stand-alone CLI for quick sample-repo scans		slowest Week
cbomkit-theia	N/A (filesystem / container focused)	Certificates, keys, secrets, OpenSSL config, java.security restrictions inside images or directories; can enrich an existing CBOM with confidence	Explicitly does not scan source code; different problem domain	Apache-2.0	Orthogonal useful contains not for discovery

Detailed Notes

cdxgen

- Official CBOM support via `cbom -t java` or `cdxgen -t <lang> --include-crypto --evidence --deep`.
- Produces `cryptographic-asset` components and can attach evidence.
- Strengths: polyglot SBOM heritage, easy CLI, already outputs schema-valid CycloneDX.
- Weaknesses relevant to us: Python detection historically incomplete (open issues about missed cryptography / Fernet / etc. usages); not designed as a lightweight regex + line-number focused scanner for educational sample repos.

sonar-cryptography

- Rule-based, AST-driven. Very high precision for the libraries it covers.
- Writes `cbom.json` when the “Cryptographic Inventory (CBOM)” rule is enabled.
- Evidence model matches what we want (`location` , `line` , `additionalContext`).

- Cost: full SonarQube installation and project setup — too heavy for a one-week PoC whose goal is a simple, self-contained Python scanner.

cbomkit-theia

- Complementary, not competing. It finds certs/keys/secrets on disk or inside images and can raise confidence levels on an existing CBOM.
- We explicitly leave container/image scanning out of Week-2 scope.

Real Observed Gaps (to be completed by Person B)

Tool	Observed on sample repo	Gap relative to QuantumShield goal
cdxgen	<i>(Person B to fill)</i>	e.g. missed hardcoded RSA key material, missing line numbers, incomplete Python API surface
sonar-cryptography	<i>(Person B to fill)</i>	e.g. setup friction, missing certain patterns
cbomkit-theia	N/A for source	Confirmed: no source scanning

Once filled, this table becomes the concrete justification paragraph in the Week-2 report: “Existing tools either lack complete language coverage, require heavy infrastructure, or solve a different problem (filesystem vs source). Therefore a purpose-built, minimal regex + CBOM writer is the fastest path to a demonstrable discovery PoC.”

Decision for QuantumShield v1

Build our own `scanner.py` + `cbom_writer.py` that: 1. Guarantees detection of the five target algorithm families on the sample repo. 2. Always emits file + line evidence. 3. Produces schema-valid CycloneDX CBOM with zero external service dependencies. 4. Can later be replaced or augmented by cdxgen / sonar-cryptography once the scoring and dashboard pillars are in place.